

JAVA . CLOUD . ORACLE . SPRINGBOOT

Oracle Cloud Infrastructure: What Java Teams Overlook

OKE, Autonomous Database, and Vault for Spring Boot workloads

Said Olano

Head of Engineering . FinTech & Digital Banking

Oracle Cloud Infrastructure: Lo que los equipos de Java pasan por alto

La mayoría de los equipos de Java optan por defecto por AWS o Azure sin siquiera cotizar Oracle Cloud Infrastructure (OCI). Es comprensible: la mentalidad colectiva no está ahí. Pero si tienes cargas de trabajo en Java, especialmente algo que toque Oracle Database, OCI tiene ventajas fáciles de pasar por alto hasta que terminas pagando tarifas de salida de datos o levantando otro clúster de Kubernetes administrado en otro lado.

El problema de asumir que "todos usan AWS"

Las decisiones de nube en la mayoría de las organizaciones de ingeniería se toman una vez, temprano, y rara vez se revisan. Eso está bien hasta que llega la factura. Dos cosas suelen sorprender a los equipos en otros proveedores: los costos de transferencia de datos de salida, y el precio de mantener cómputo siempre activo para cargas que no lo necesitan. Ninguno es exclusivo de un proveedor en particular, pero el modelo de precios de OCI —plano, predecible y notablemente más barato en salida de datos— vale la pena revisarlo si no lo has cotizado desde tu última comparación de proveedores.

Lo que realmente es útil para un stack de Java

Varios servicios de OCI se alinean directamente con lo que un equipo típico de Spring Boot / microservicios ya necesita:

- **Autonomous Database** — una base de datos Oracle que se autoparcha y autoajusta. Si tu aplicación ya se comunica con Oracle vía JDBC, esto elimina una carga considerable de trabajo de DBA.
- **Container Engine for Kubernetes (OKE)** — una oferta de Kubernetes administrada que es un destino directo para los mismos Helm charts y manifiestos que ya usas en otro lado.

- **OCI Functions** — serverless, construido sobre el proyecto open-source Fn Project, lo que significa que es portable en lugar de un callejón sin salida propietario.
- **Vault** — gestión centralizada de secretos y llaves, con la misma forma que AWS Secrets Manager o HashiCorp Vault, si prefieres no montar el tuyo propio.

Nada de esto requiere reescribir nada. Un servicio de Spring Boot que ya externaliza su configuración de datasource puede apuntar a una instancia de Autonomous Database con un cambio de una línea:

```
spring:
  datasource:
    url: jdbc:oracle:thin:@myatp_high?TNS_ADMIN=/opt/oracle/wallet
    username: ${DB_USER}
    password: ${DB_PASSWORD}
    driver-class-name: oracle.jdbc.OracleDriver
```

La conexión basada en wallet maneja el TLS y el failover por ti, lo cual es una cosa menos que armar a mano en un HikariConfig.

Dónde se queda corto

El ecosistema y el tamaño de la comunidad de OCI no se acercan a los de AWS. Menos módulos de Terraform, menos respuestas en Stack Overflow, menos integraciones de terceros listas para usar. La presencia multi-región también es más delgada, así que si tus requisitos de latencia son globales y específicos, revisa la cobertura de regiones antes de comprometerte. Y si tu equipo tiene años de herramientas y memoria muscular específicas de AWS, el costo de migrar por completo rara vez se justifica solo por el ahorro en salida de datos.

Dónde vale la pena evaluarlo

El caso para OCI es más fuerte cuando ya eres un equipo que usa Oracle Database, cuando tus cargas de trabajo son de lectura intensiva con mucho tráfico de salida, o cuando estás construyendo algo desde cero y la previsibilidad de costos importa más que la amplitud del ecosistema. Rara vez es un reemplazo total de AWS o Azure — más a menudo se gana un lugar en una estrategia multi-nube para las cargas específicas en las que realmente es bueno.

¿Ya evaluaste OCI para alguna carga de trabajo en Java, o AWS/Azure sigue siendo el default incuestionable en tu equipo?